'has a'
Composition
Aggregation
Association

'is a'
Generalisation
Realisation

'Use a'
Dependency

GameOutcome
<<enumeration>>
IN_PROGRESS
VICTORY
DEFEAT

GameConfig
<<Entity>>
-maxTurns: int = 18
-tableCount: int = 2
-victoryRevenueCents: int = 4500
-victoryPaidCustomers: int = 4
-victorySatisfaction: double = 60
-FIRST_ARRIVAL_TURN: int = 1
-ARRIVAL_INTERVAL: int = 3
-TOTAL_ARRIVALS: int = 6
+getMaxTurns(): int
+getTableCount(): int
+getVictoryRevenue(): int
+getVictoryPaidCustomers(): int
+getVictorySatisfaction(): double
+isArrivalTurn(turn: int): boolean

GameResult
<<Entity>>
-revenueCents: int
-paidCount: int
-satisfactionTotal: int
-outcome: GameOutcome
-servedCount: int
-abandonedCount: int
+recordPayment(priceCents: int, satisfaction: int): void
+recordServed(): void
+recordAbandonment(): void
+calculateAverageSatisfaction(): double
+evaluate(cfg: GameConfig): GameOutcome
+isVictory(): boolean
+getRevenue(): int
+getPaidCount(): int
+getSatisfactionTotal(): int
+getOutcome(): GameOutcome
+getServedCount(): int
+getAbandonedCount(): int

UserInterface
<<Boundary>>
-in: Scanner
-inputEnded: boolean
+UserInterface()
+UserInterface(in: Scanner)
+displayState(turn: int, maxTurns: int, restaurant: Restaurant, result: GameResult): void
+promptDecision(options: List<Decision>): Decision
+displayError(message: String): void
+displayMessage(message: String): void
+displayResult(result: GameResult, cfg: GameConfig, turnsUsed: int): void
-waitOption(options: List<Decision>): Decision
-averageText(result: GameResult): String
-money(cents: int): String

Decision
<<Entity>>
-type: Decision.Type
-customer: Customer
-table: Table
-order: Order
-description: String
-Decision(type: Type, customer: Customer, table: Table, order: Order, description: String)
+seat(c: Customer, t: Table): Decision
+takeOrder(c: Customer, t: Table): Decision
+serve(o: Order, t: Table): Decision
+prepare(o: Order, t: Table): Decision
+doNothing(role: String): Decision
+getType(): Decision.Type
+getCustomer(): Customer
+getTable(): Table
+getOrder(): Order
+getDescription(): String
+toString(): String

Decision.Type
<<enumeration>>
SEAT
TAKE_ORDER
SERVE
PREPARE
WAIT

idea is for **GameController** to manage the round logic and for **TurnController** to manage execution of each individual turn

GameController
<<Control>>
-currentTurn: int
-config: GameConfig
-ui: UserInterface
-result: GameResult
-turnController: TurnController
+GameController(config: GameConfig, ui: UserInterface)
+startGame(): void
+runGame(): void
+getResult(): GameResult

TurnController
<<Control>>
-config: GameConfig
-restaurant: Restaurant
-result: GameResult
-ui: UserInterface
-waiter: Waiter
-chef: Chef
-cashier: Cashier
-customerFactory: CustomerFactory
-nextCustomerId: int
+TurnController(config: GameConfig, restaurant: Restaurant, result: GameResult, ui: UserInterface)
+executeTurn(turn: int): void
-processArrivals(turn: int): void
-processWaiterAction(d: Decision, turn: int): void
-processChefAction(d: Decision, turn: int): void
-processEating(turn: int): void
-processPayment(): void
-processAbandonment(): void
-buildWaiterOptions(): List<Decision>
-buildChefOptions(): List<Decision>
-abandon(c: Customer): void
-money(cents: int): String

processWaiterAction and **processChefAction** are dependent on the decisions from the manager (i.e the player). Each decision is one of the options built by **buildWaiterOptions** / **buildChefOptions**.

Order of steps inside **executeTurn**: arrivals, display, Waiter decision, Chef decision, processPayment, processEating, processAbandonment. Payment runs before eating so a customer who becomes READY_TO_PAY pays on the next turn.

Composition
Restaurant creates and owns its 2 Tables and its WaitingQueue. They have no life outside the Restaurant.

Restaurant
<<Entity>>
-tables: List<Table>
-waitingQueue: WaitingQueue
-menu: List<MenuItem>
+Restaurant(tableCount: int)
+getAvailableTable(): Table
+getWaitingQueue(): WaitingQueue
+getOccupiedTables(): List<Table>
+findTableOf(c: Customer): Table
+getTables(): List<Table>
+getMenu(): List<MenuItem>

Waiter
<<Control>>
-restaurant: Restaurant
+Waiter(restaurant: Restaurant)
+seatCustomer(c: Customer, t: Table): void
+takeOrder(c: Customer): Order
+serve(o: Order, turn: int): void

Chef
<<Control>>
+prepareOrder(o: Order, turn: int): void

Cashier
<<Control>>
+processPayment(o: Order, r: GameResult): void

CustomerFactory
<<Control>>
-VIP_ARRIVAL: int = 2
-CRITIC_ARRIVAL: int = 4
+createCustomer(id: int): Customer

Arrival schedule (Stage 2)
Still six arrivals on turns 1, 4, 7, 10, 13, 16: Regular, VIP, Regular, Critic, Regular, Regular. CustomerFactory is the only class that names the concrete types.

Table
<<Entity>>
-tableNumber: int
-occupant: Customer
+Table(tableNumber: int)
+isAvailable(): boolean
+seatCustomer(c: Customer): void
+release(): void
+getTableNumber(): int
+getOccupant(): Customer

WaitingQueue
<<Entity>>
-customers: Queue<Customer>
+enqueue(c: Customer): void
+dequeueNext(): Customer
+remove(c: Customer): boolean
+isEmpty(): boolean
+getSize(): int
+getCustomers(): List<Customer>

Customer
<<Entity>>
-STARTING_SATISFACTION: int = 100
-REGULAR_SATISFACTION_LOSS: int = 8
-id: int
-satisfaction: int
-status: CustomerStatus
-turnServed: int
-order: Order
+Customer(id: int)
+getSatisfactionLoss(): int
+reduceSatisfaction(): void
+chooseMenuItem(menu: List<MenuItem>): MenuItem
+calculateBill(item: MenuItem): int
#getTypeLabel(): String
+setServed(turn: int): void
+getTurnServed(): int
+isWaitingForService(): boolean
+getId(): int
+getSatisfaction(): int
+getStatus(): CustomerStatus
+setStatus(status: CustomerStatus): void
+getOrder(): Order
+setOrder(order: Order): void
+toString(): String

CustomerStatus
<<enumeration>>
WAITING
SEATED
ORDERED
EATING
READY_TO_PAY
PAID
ABANDONED

Order
<<Entity>>
-customer: Customer
-status: OrderStatus
-preparationRemaining: int
-priceCents: int
-turnReady: int
-turnServed: int
-item: MenuItem
+Order(customer: Customer, item: MenuItem)
+advancePreparation(): void
+isFullyPrepared(): boolean
+markReady(turn: int): void
+markServed(turn: int): void
+markPaid(): void
+markCancelled(): void
+getCustomer(): Customer
+getItem(): MenuItem
+getPrice(): int
+getStatus(): OrderStatus
+getPreparationRemaining(): int
+getTurnReady(): int
+getTurnServed(): int

OrderStatus
<<enumeration>>
PREPARING
READY
SERVED
PAID
CANCELLED

Polymorphism
TurnController and Waiter only use the Customer type. VIPCustomer and CriticCustomer override getSatisfactionLoss(), calculateBill() and chooseMenuItem(), so the right behaviour is chosen at run time with no type checks. Order calls customer.calculateBill(item) when it is created, so the price is locked when the order is taken.

Encapsulation
Order keeps status and preparationRemaining private. Other classes can only change them through Order's own methods (advancePreparation(), markReady(), markServed(), markPaid(), markCancelled()); the controllers call these in the correct turn order.

When the manager chooses **take order** for a SEATED customer, the Waiter creates the order and its default state is set to PREPARING. When the order is served, the customer status becomes EATING.

MenuItem
<<Entity>>
-name: String
-basePriceCents: int
-preparationUnits: int
+MenuItem(name: String, basePriceCents: int, preparationUnits: int)
+getName(): String
+getPrice(): int
+getPrepUnit(): int

Dependencies that appear only as attribute or parameter types are not drawn as separate lines, e.g. GameController -> GameConfig, TurnController -> GameConfig / GameResult, UserInterface -> Restaurant / GameConfig, Cashier -> GameResult, Decision -> Customer / Table / Order.

VIPCustomer
<<Entity>>
-VIP_SATISFACTION_LOSS: int = 5
-VIP_DISCOUNT_RATE: double = 0.10
+VIPCustomer(id: int)
+getSatisfactionLoss(): int
+calculateBill(item: MenuItem): int
#getTypeLabel(): String

CriticCustomer
<<Entity>>
-CRITIC_SATISFACTION_LOSS: int = 12
+CriticCustomer(id: int)
+getSatisfactionLoss(): int
+chooseMenuItem(menu: List<MenuItem>): MenuItem
#getTypeLabel(): String

Critic order preference
A critic always orders the most expensive item on the menu (ties: first listed). With the current menu that is Pasta, 2 preparation units.

uses
uses
<<create>>
<<create>>
<<create>>
places
menu
1
0..1
0..*
2
3..*